

Network Programming HW1 Report: Two-Player Online Game

Zheng Wu Qian (112550200)

1. System Architecture

Overview

The system consists of three major components:

1. **Lobby Server (TCP-based)** – responsible for user registration, authentication, and maintaining persistent records.
2. **Peer-to-Peer (P2P) Communication Layer** – handles player discovery (UDP) and gameplay session (TCP).
3. **Database Layer (JSON files)** – stores users' credentials and statistics persistently.

Lobby Server

The lobby server (`lobby_server.py`) runs on TCP port 12000. It accepts multiple simultaneous connections using Python's `threading` library and `JsonLineSocket` for structured JSON-based communication: `contentReference[oaicite:0]index=0`. Core responsibilities:

- **Account registration / login** via `REGISTER` and `LOGIN` message types.
- **Statistics maintenance** to update wins, losses, and logins.
- **Session control** through `LOGOUT` to avoid duplicate logins.
- **Persistence**: all user data is stored in `users.json` and `stats.json`, allowing recovery after server restarts.

A global lock protects shared structures to ensure safe concurrent writes.

Peer-to-Peer Architecture

After login, players establish direct P2P communication:

- **UDP (Invitation phase)** – Player A scans and invites Player B.
- **TCP (Game phase)** – After Player B accepts, Player A creates a TCP server and Player B connects to it.

This two-stage protocol provides efficient discovery with reliable game synchronization.

Database Layer

Two JSON files form the persistent database:

- **users.json** – contains username–password pairs.
- **stats.json** – records wins, losses, and login counts.

Example content:

```
users.json:
{"playerA":"pass123","playerB":"123"}
```

```
stats.json:
{"playerA":{"wins":1,"loses":0,"logins":8}}
```

These files ensure durability across sessions and server restarts.

2. Communication Between Players and Server

Data Format and Transport

All components communicate using UTF-8 encoded JSON strings. Each message is terminated by a newline character (`\n`) to mark message boundaries. The helper functions `send_json_line()` and `recv_json_line()` in `common.py` abstract this logic: [contentReference\[oaicite:1\]in](#)

A. Lobby Server Communication (TCP)

The lobby server accepts the following JSON message types:

- **REGISTER:**

```
{"type":"REGISTER","username":"playerA","password":"pass123"}
→ {"ok":true,"msg":"REGISTER_SUCCESS"}
```

Handles account creation; if the user already exists, returns:

```
{"ok":false,"msg":"USER_EXISTS"}
```

- **LOGIN:**

```
{"type":"LOGIN","username":"playerA","password":"pass123"}
→ {"ok":true,"msg":"LOGIN_SUCCESS"}
```

Validates credentials and ensures no duplicate sessions are active.

- **REPORT:**

```
{"type": "REPORT", "username": "playerA", "delta": {"wins": 1}}  
→ {"ok": true, "msg": "REPORT_OK"}
```

Periodically updates player statistics such as wins, losses, and login counts.

- **LOGOUT:**

```
{"type": "LOGOUT", "username": "playerA"}  
→ {"ok": true, "msg": "LOGOUT_OK"}
```

Ensures proper cleanup and prevents duplicate login issues.

B. UDP Discovery and Invitation

Player A discovers Player B instances via UDP:

1. **Discovery** A sends a probe message to a port range (18000–18020):

```
{"type": "DISCOVER"}
```

Each Player B replies if idle:

```
{"type": "HERE", "username": "playerB", "udp_port": 18000, "status": "waiting"}
```

2. **Invitation** Once discovered, Player A sends:

```
{"type": "INVITE", "from": "playerA"}
```

Player B can choose to accept or reject:

```
{"type": "INVITE_REPLY", "accept": true}
```

C. Establishing TCP Connection

When Player B accepts the invitation:

1. Player A creates a TCP server using `bind_with_retry()` to handle potential port conflicts: `contentReference[oaicite:2]index=2`.
2. Player A sends connection info back to Player B via UDP:

```
{"type": "TCP_INFO", "host": "127.0.0.1", "port": 19000}
```

3. Player B connects to the specified host and port and starts the game session.

D. In-Game TCP Communication

Once connected, both players exchange structured game state messages:

- **STATE** – Full board, turn, and winner.
- **MOVE** – Player's chosen column index.
- **ERROR** – Invalid move or format.
- **BYE** – Session termination notice.

Example TCP sequence:

```
A → {"type": "STATE", "board": [...], "turn": "A"}
B → {"type": "MOVE", "col": 3}
A → {"type": "STATE", "board": [...], "turn": "B"}
```

3. Gameplay and Rules

Game Type: Connect Four

The game (`connect4.py`) implements a 6×7 grid-based Connect Four:contentReference[oaicite:3]index=

Rules

1. Players alternate dropping their stones (1 for A, 2 for B).
2. The stone occupies the lowest available position in the chosen column.
3. A player wins when four consecutive stones are aligned horizontally, vertically, or diagonally.
4. If the board fills with no winner, the game results in a draw.

Game Flow

- **Start:** Player A starts the TCP server; Player B connects.
- **Turns:** The server broadcasts the updated game state after each move.
- **End:** When a winner or draw is detected, a final **STATE** message is sent, followed by a **BYE**.

Ending Conditions

- **Winner Found:** Detected by `_check_win()` function.
- **Draw:** All columns filled.
- **Disconnect:** Gracefully handled by sending **BYE**.

Error and Exception Handling

- **Invalid Input:** The system catches input errors and re-prompts without breaking the session.
- **Port Conflicts:** The function `bind_with_retry()` automatically retries on incremented ports.
- **Duplicate Login:** The lobby rejects re-login attempts from already active users.
- **Disconnection Recovery:** If one player leaves mid-session, the other is safely notified and can return to waiting mode.

Reporting Game Results

After each match, both players update their win/loss statistics through:

```
{"type": "REPORT", "username": "playerA", "delta": {"wins": 1}}
```

This maintains long-term records across multiple sessions: `contentReference[oaicite:4]index=4`.

4. Feature Coverage Summary (Mapping to Grading Rubric)

A. Lobby Server and Account Operations

- **Account Registration:** Implemented in REGISTER handler with JSON response.
- **Account Login:** Implemented in LOGIN handler; updates `online` set.
- **Database:** Data stored in `users.json` and `stats.json`.
- **Exception Handling:** Handles invalid inputs, duplicate registration, and non-existent users.
- **Periodic Reporting:** Players send REPORT updates after games to record cumulative stats.
- **Duplicate Login Handling:** Prevents multiple sessions per account; proper logout removes user.

B. Game Invitation and Connection

- **UDP Scanning:** Player A uses `udp_discover()` to detect waiting Player B instances.
- **Player B Waiting:** Player B listens on ports 10000 with `udp_wait_loop()`.
- **Invitation Handling:** Invitation and acceptance flow implemented using JSON over UDP.
- **TCP Establishment:** Player A sends TCP info via UDP, Player B connects successfully for gameplay.

C. Online Gaming

- **Content and Interaction:** Fully functional Connect Four with multiple turn exchanges.
- **Game Flow and Termination:** The game ends gracefully with winner/draw detection.
- **Exception Handling:**
 - Automatic port retry mechanism.
 - Input validation and in-session error recovery.
 - Graceful handling of disconnections and duplicate logins.

5. Conclusion

This project successfully implements a full two-player online game system meeting all HW1 requirements:

- Persistent and concurrent account management via lobby server.
- Efficient UDP discovery and invitation protocol.
- Reliable TCP-based gameplay with error handling.
- Automated statistics reporting and persistence.

The architecture demonstrates an integrated understanding of TCP/UDP programming and real-time multiplayer synchronization.